Searching Algorithms

1. Linear Search (Sequential Search)

Introduction

- Linear Search is a searching technique in which elements are checked one by one from the beginning until the required element is found or the list ends.
- It works on both **sorted and unsorted data**.
- It can be applied to arrays and linked lists.

Working

- Start searching from the first element.
- Compare each element with the target value.
- If a match is found, return its index/position.
- If the end of the collection is reached without a match, return `-1`.

Important Conditions (Java)

- Element found: `arr[i] == key`
- Element not found: Return `-1`.
- Best case: Element is present at the first position.
- Worst case: Element is present at the last position or is absent.

Time and Space Complexity

Case	Time Complexity
Best Case	O(1)
Average Case	O(n)
Worst Case	O(n)

SUNBEAM

- Space Complexity: **O(1)** for iterative implementation.
- For `n` elements:
  - Best case: 1 comparison.
  - Worst case: n comparisons.
  - Average case (successful search with equally likely positions): `(n + 1) / 2` comparisons.

2. Binary Search

Introduction

- Binary Search is an efficient searching technique that repeatedly divides the search space into two halves.
- It requires the input data to be **sorted**.
- It works by comparing the target element with the middle element.
- It can be implemented using iterative or recursive approaches.
- It is most suitable for arrays that support random access.

Working

- Initialize two variables:
  - `low = 0`
  - `high = arr.length - 1`
- Calculate the middle index.
- Compare the middle element with the target:
  - If equal, element is found.
  - If target is smaller, search the left half.
  - If target is greater, search the right half.
- Repeat until the element is found or `low > high`.

Important Conditions (Java)

- Middle index calculation:
  - `mid = low + (high - low) / 2`

SUNBEAM

    - This avoids potential integer overflow compared to `(low + high) / 2`.
- If `arr[mid] == key`:
    - Element found.
- If `arr[mid] > key`:
    - `high = mid - 1`
- If `arr[mid] < key`:
    - `low = mid + 1`
- Search terminates when:
    - `low > high`
- For a recursive implementation, the base condition for an unsuccessful search is `low > high`.

Time and Space Complexity

Case	Time Complexity
Best Case	O(1)
Average Case	O(log n)
Worst Case	O(log n)

- Space Complexity:
    - Iterative Binary Search: **O(1)**
    - Recursive Binary Search: **O(log n)** due to recursion stack.
- Each iteration reduces the search space approximately by half.
- Maximum number of iterations is proportional to $\log_2(n)$.

3. Linear Search vs Binary Search

SUNBEAM

Parameter	Linear Search	Binary Search
Searching method	Sequential	Divide and Conquer
Data requirement	Sorted or unsorted	Must be sorted
Best case	O(1)	O(1)
Average case	O(n)	O(log n)
Worst case	O(n)	O(log n)
Space (iterative)	O(1)	O(1)
Comparisons	Up to n	Approximately $\log_2(n)$
Implementation	Simple	Comparatively complex
Linked List	Suitable	Not efficient
Random access	Not required	Important for efficiency

4. Advantages and Disadvantages

Linear Search

Advantages

- Simple and easy to implement.
- Works on sorted and unsorted data.
- Does not require additional memory.
- Suitable for small datasets.
- Works efficiently with linked lists.

Disadvantages

- Slow for large datasets.
- May require checking every element.

SUNBEAM

- Less efficient than Binary Search for large sorted arrays.

Binary Search

Advantages

- Very efficient for large sorted datasets.
- Requires fewer comparisons than Linear Search.
- Iterative implementation requires O(1) auxiliary space.
- Reduces the search space by half in every iteration.

Disadvantages

- Requires sorted data.
- Maintaining sorted data can involve additional overhead.
- Not efficient for linked lists because accessing the middle element requires traversal.
- Slightly more complex to implement.

5. Applications

Linear Search

- Searching elements in small arrays.
- Searching in unsorted datasets.
- Searching elements in linked lists.
- Finding an element in a sequential collection.
- Searching in datasets where sorting is not practical.

Binary Search

- Searching in sorted arrays.
- Searching in dictionaries and phone directories.
- Finding elements in sorted databases or collections.
- Finding lower bound and upper bound positions.

SUNBEAM

- Finding square roots using Binary Search.
- Binary Search on Answer problems (optimization problems with a monotonic search condition).

6. Important MCQ Points

- Linear Search works on both sorted and unsorted data.
- Binary Search requires sorted data.
- Linear Search worst-case time complexity: **O(n)**.
- Binary Search worst-case time complexity: **O(log n)**.
- Best-case complexity of both algorithms: **O(1)**.
- Iterative implementations of both algorithms require **O(1) auxiliary space**.
- Recursive Binary Search requires **O(log n) stack space**.
- Binary Search follows the **Divide and Conquer** approach.
- Linear Search is also called **Sequential Search**.
- Binary Search eliminates approximately half of the remaining elements in every iteration.
- For 1024 elements, Binary Search requires at most approximately 10 comparisons (with the usual balanced search process).
- Binary Search is not efficient on a linked list because random access is unavailable.
- If duplicate elements exist, ordinary Binary Search may return any matching occurrence.
- To find the first or last occurrence of a duplicate element, modified Binary Search is required.
- Binary Search can be applied to descending sorted data by reversing the comparison conditions.
- Sorting an unsorted dataset before Binary Search introduces additional time complexity.

SUNBEAM